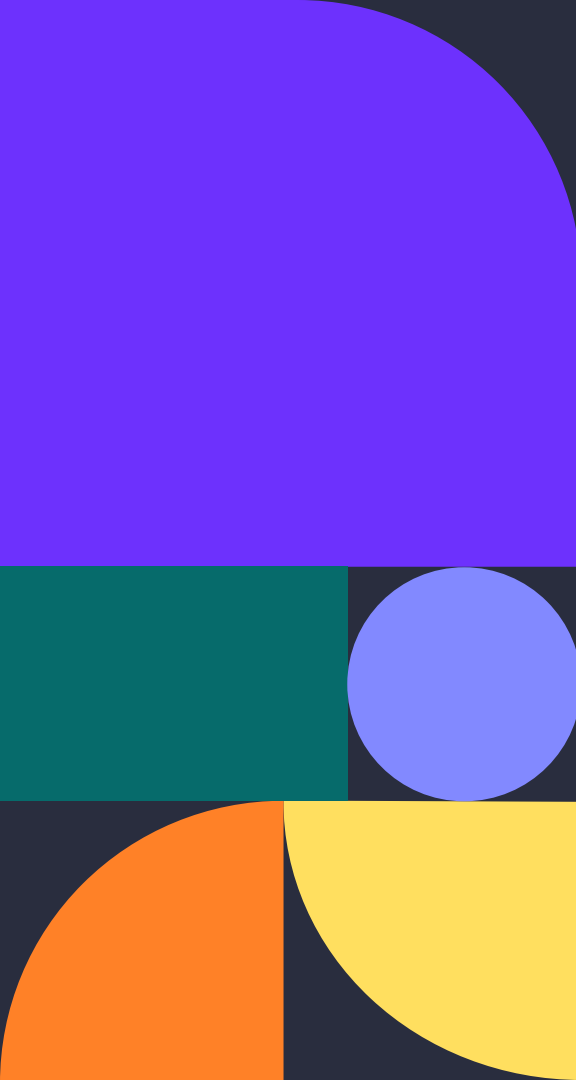




Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning

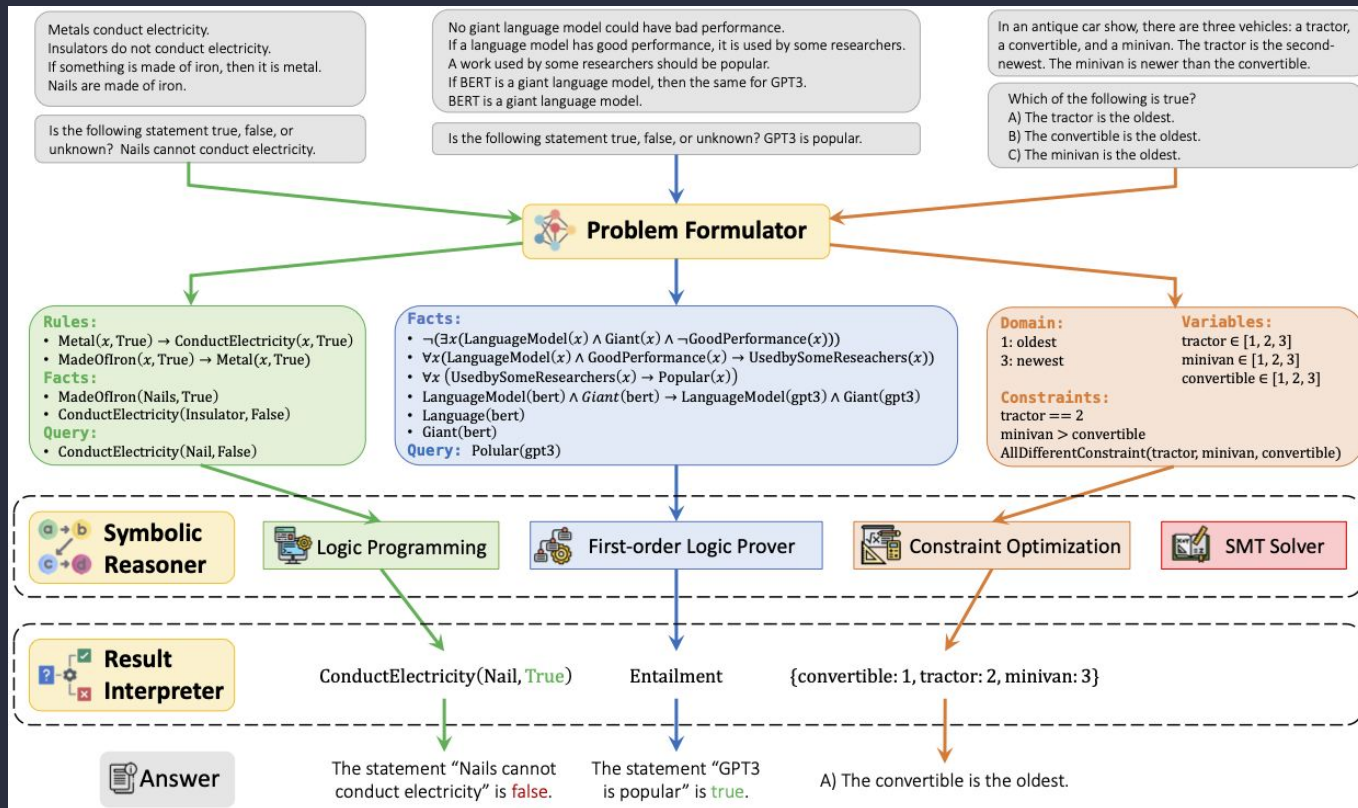
Atul Bhat

What's the problem?

- LLMs are great at interacting with most use cases but unreliable at pure logic (e.g., deduction, puzzles, LSAT questions)
- Chain-of-thought prompting is better than nothing but sometimes also skips logical steps, which could make it land at an unreasonable conclusion
- Ultimately, the way LLMs work is based on inference plausible text, not actually reasoning

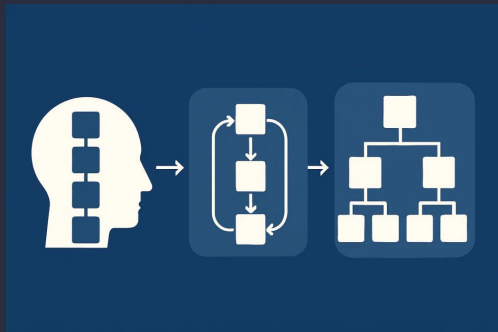
The Solution

LLMs are great at NLP, formatting how humans converse into a structured format. Solvers are deterministic, meaning that instead of hoping an LLM can logically come to the correct conclusion, solvers can guarantee it. A keyword in this paper is "faithful reasoning", meaning that not only do you need to the correct answer but also have correct steps that lead to that answer. LLMs often hallucinate and make logical errors during the process of generating an answer, so integrating both Logic and LLM together offers the accessibility and flexibility of an LLM with the precision of logic-driven solvers.



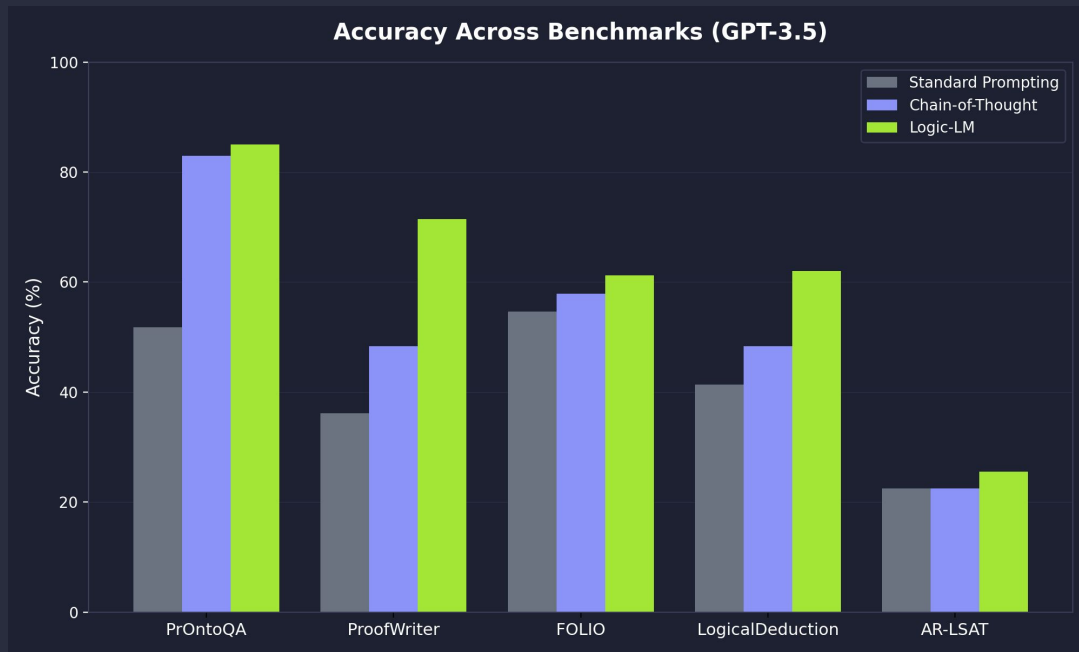
We have talked about how the LLM and the solver would work together. It is important to note the loop, in which if the solver throws an error, the LLM revises its formalization, or syntax, using the error message, until it works

Results



Benchmarks

- ProofWriter
- PrOntoQA
- FOLIO
- LogicalDeduction
- AR-LSAT



**+39.2% over
standard LLM
prompting**

**+18.4% over
chain-of-thought
prompting**



Takeaways

The successful results of this study show that Logic-LM is simply a better engine together than their parts alone. The loop keeps errors at bay, and help the success of the engine as more and more epochs go by. While there is the limitation that everything relies on the initial translation, the moral of the story is that pairing LLMs with logic can help patch holes of things they were previously bad at.

Thank you